Formal Market Microstructure in U.S. Equities II

Observable Conformance, Latent-State Identification, and Clock Uncertainty

Miquel Noguer i Alonso
Artificial Intelligence Finance Institute

September 5, 2026

Paper DOI: 10.5281/zenodo.22392230
github.com/MiquelNoguerAlonso/Formal-Verification

Abstract

This paper develops a formal account of what market data can establish about the execution of an allocation rule. A correct matching algorithm and an observable matching decision are different achievements: a trade total can conceal a priority violation, an unknown wheel position can leave several explanations, and uncertain event order can make different executions observationally compatible. The analysis connects the verified price–time and persistent parity rules of the companion paper to three observation models for U.S. equities.

First, transferring a positive fill between resting orders preserves its aggregate total while changing the vector of fills attached to those orders. A total-only projection therefore cannot identify that transfer, whereas an exact depth observation rejects it against an authenticated priority queue. Second, a finite wheel checker searches every rotation and positive allowance in a declared initial family. Its search has exactly mL entries for m participants and lot size L; acceptance exhibits an explaining state, and additional executions can only remove candidates. Worked examples show both immediate identification and persistent ambiguity. Third, a clock uncertainty checker compares fills by fixed order identity across admissible queue permutations. Acceptance grows with the uncertainty threshold. For two orders whose swapped allocations differ, masking occurs exactly when their timestamp gap lies within that threshold. An exponential-gap model supplies an analytic masking probability, illustrated by a reproducible simulation.

Lean 4.22.0 checks 29 theorem declarations, none depending on classical choice, within an artifact containing 391 declarations. The mathematical guarantees concern the supplied model and inputs. They do not authenticate a feed, reconstruct hidden liquidity, or establish production compliance. The paper specifies the reconstruction and reporting obligations needed to apply its checkers, separates logical compatibility from statistical inference, and provides source code, reproducible figures, and worked instances. No exchange data are analyzed; empirical application requires validated feed reconstruction.

Keywords: U.S. equities; formal verification; market data; conformance; order identity; latent state; clock resolution; Lean 4.

Contents

1 Introduction

An execution can satisfy the published matching rule and still be impossible for an outside observer to verify. The obstacle may be absent order identifiers, an unknown parity pointer, or incomplete evidence about priority. A proof about an allocation algorithm does not remove those information gaps. It provides a precise rule against which the remaining explanations can be checked.

The companion paper *Foundations of Formal Market Microstructure in U.S. Equities* (Noguer i Alonso, 2026) establishes executable semantics for strict price–time priority and a parity wheel that persists its pointer and unused lot allowance across executions. This paper asks what an observer can infer when only part of the execution record is available. Its three results have different strengths:

1. A total-preserving queue jump is invisible to a total-only projection and rejected when the exact priority queue and order-aligned fills are available (Theorems 3.1–3.2).

2. A wheel checker searches a finite family of initial phase states. Acceptance is equivalent to existence of an explaining member, and consistency survives passage to a prefix (Theorems 4.4–4.7). The family has mL list entries; this count alone is not a behavioral information bound.

3. A checker for uncertain event order preserves order identities when trying alternative queues. Acceptance is monotone in the uncertainty threshold. A nearby adjacent swap supplies a sufficient masking condition; a separate two-order theorem establishes an exact threshold when swapping changes the labelled allocation (Theorems 5.4–5.5).

These results support a model-relative audit: rejection excludes the declared explanations, provided the inputs and admissible family are correct. Acceptance establishes compatibility with at least one explanation. Neither conclusion authenticates the input data or establishes legal compliance. Section 6 makes these obligations explicit.

Related work. Timestamp and liquidity measurement problems are studied by Holden and Jacobsen (2014); SIP latency and discrepancies with direct feeds are examined by Bartlett and McCrary (2019) and Ding et al. (2014). Message-level analyses include Aquilina et al. (2022) and Kirilenko et al. (2017). These studies motivate careful reconstruction; the present contribution specifies finite logical compatibility checks rather than estimating their empirical incidence. Passmore and Ignatovich (2017) develop formal verification for financial algorithms, with subsequent exchange-matching work described by Brennan (2024). The rationing background is developed by Moulin (2000) and Thomson (2003). The distinction between observed order and physical time also connects to Lamport (1978).

Scope. The analysis concerns idealized allocation at a fixed price, with a declared eligible queue and rule. It does not model the entire trading system. In particular, finite timestamps do not automatically make exchange priority ambiguous: a sequenced message stream can preserve order within a timestamp. The clock model applies only to uncertainty remaining after trustworthy sequence and priority evidence has been used. No TAQ or ITCH dataset is analyzed here.

2 Objects and observation contracts

Quantities are natural numbers of shares. A resting queue at one price is $Q = (q_1, \dots, q_n)$ in priority order. An incoming quantity x produces an allocation $A = (a_1, \dots, a_n)$, indexed by that same order

list. The package defines $\mathrm{PT}(Q, x) = \texttt{priceTime}\, Q\, x$ as sequential filling and `conformsPT` as the Boolean equality test against this vector. The companion paper characterizes this allocation by feasibility and strict serial priority.

For the parity wheel, `WState` consists of a cyclically ordered list of triples (id, allocated, remaining) and an unused lot allowance. The quantity L is the allocation quantum. The function `runW L n st x` executes an incoming quantity with fuel n; `initW L R` initializes from participant claims R. A partial visit retains its pointer and residual allowance. A completed visit advances the pointer and resets the allowance to L.

Definition 2.1 (Observation projections)**.** For one allocation vector A, define:

- the *total-only projection*, $\texttt{tapeView}(A) = \sum_i a_i$;
- the *exact depth projection*, $\texttt{depthView}(A) = A$, aligned to authenticated order identities and priority;
- a *clock uncertainty model*, in which the labelled fills are fixed but admissible priority queues vary over a declared swap neighbourhood (Section 5).

For the wheel, `execFills` reports the change in each participant's allocated counter, in a fixed external identifier order.

The first projection deliberately retains only aggregate quantity. A real trade record contains additional fields. Daily TAQ includes trade and quote products derived from the consolidated feeds (NYSE, 2022, pp. 4, 17–20). The invisibility result below holds with other observed fields fixed; it does not equate the entire TAQ product with a single sum. At the order level, Nasdaq TotalView-ITCH provides order references for displayed-book events, but its architecture also supplies sequenced messages (Nasdaq, n.d., pp. 3–4, 13–15). Nanosecond timestamp precision is therefore not, by itself, a license to permute messages.

All positional comparisons require matching lengths and correct alignment. In labelled comparisons, order identifiers must be unique, and the external identifier list must contain every relevant order exactly once. The definitions are total Lean functions even on malformed inputs, but the interpretation as an order audit requires these input conditions. Hidden interest, eligibility exceptions, and priority-changing events must be represented or excluded by the observation contract.

3 Tape versus depth

A queue jump transfers t shares of fill from an earlier position to a later position. Using $+\!\!+$ for list concatenation, write

$$H = U +\!\!+ [a] +\!\!+ V +\!\!+ [b] +\!\!+ W,$$
$$J = U +\!\!+ [a - t] +\!\!+ V +\!\!+ [b + t] +\!\!+ W.$$

Subtraction is truncated on $\mathbb{N}$; the hypothesis $t \le a$ ensures an actual transfer. The definitions are `honest` and `jumped`. If feasibility of the altered execution is also required, the later order must have at least t shares of unused capacity.

Theorem 3.1 (Total invisibility; `jump_tape_invisible`)**.** *For all lists U, V, W and $t \le a$,* $\texttt{tapeView}(J) = \texttt{tapeView}(H)$.

Theorem 3.2 (Depth rejection; `jump_depth_detected`)**.** *If* $\mathrm{PT}(Q, x) = H$ *and* $0 < t \le a$*, then* $\texttt{conformsPT}\, Q\, J\, x = \texttt{false}$.

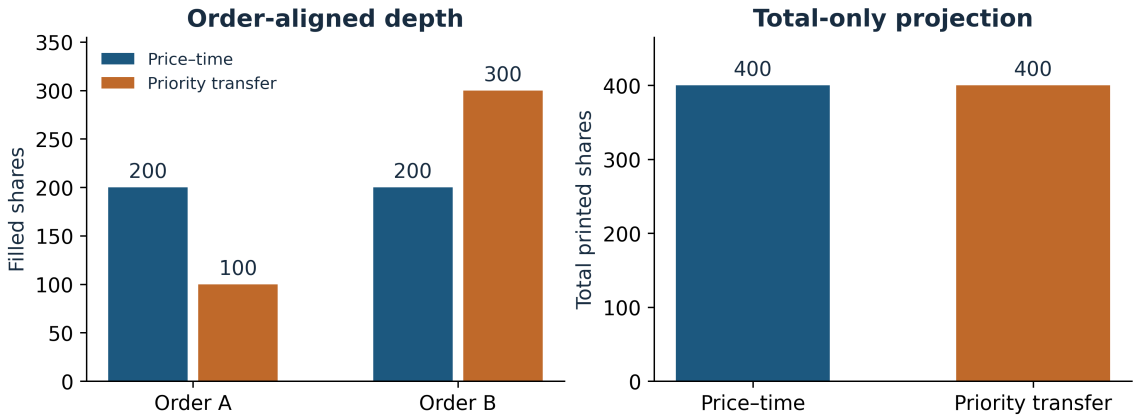

Figure 1: Claims $(200, 300)$ and incoming quantity 400. The price–time allocation $(200, 200)$ and the feasible jump $(100, 300)$ have identical totals. Keeping order labels reveals the difference. Values are exported from Lean.

Proposition 3.3 (Honest acceptance; `honest_conforms`)**.** *For every Q, x,*

$$\texttt{conformsPT}\, Q\, \mathrm{PT}(Q, x)\, x = \texttt{true}.$$

Corollary 3.4 (What the total identifies)**.** *Two allocations have the same total-only observation exactly when their sums agree. A price–time allocation has total* $\min(x, \sum_i q_i)$*. Consequently, for fixed Q, x, every feasible allocation attaining this total has the same total-only observation, regardless of priority.*

Proof. The first claim is the definition. The total identity follows from `priceTime_feasible` and the input bound `priceTime_sum_le`; equivalently, sequential filling exhausts either the incoming quantity or the queue. The last claim is substitution into `tapeView`. □

Figure 1 isolates the information loss. It does not imply that every study using trades alone measures only feasibility: prices, timing, conditions and external evidence may answer other questions. It establishes that this total-only observation cannot distinguish the displayed priority transfer. Even checking the expected aggregate requires knowledge of the eligible queue and incoming quantity.

4 Wheel conformance with a latent phase

A persistent parity wheel can give different next fills from different phase states. The companion paper proves separation results on nondegenerate states and exact finite-family information bounds under specified observation contracts. It also derives the reachable allowance identity

$$\text{allowance} = L - (A_p \bmod L)$$

when the pointer p has a live head and A_p is its authenticated cumulative allocation from initialization. Thus the allowance is not independent extra information when that historical counter is known. Here the audit window starts with known remaining claims and cyclic order, but without that history or a known phase.

Definition 4.1 (Candidate family; `candidates`)**.** For m claims R and positive lot L, take every rotation S of the participant list of $\texttt{initW}\, L\, R$, and every allowance $a \in \{1, \ldots, L\}$. The resulting list of states $\langle S, a \rangle$ is $\texttt{candidates}\, L\, R$.

Allocated counters are initialized to zero as counters *within the audit window*. For a candidate with allowance below L, this zero is a rebasing convention with unknown historical offsets; it is not a claim of reachability from zero historical fills. Where authenticated history is available, the allowance identity or other evidence should restrict the admissible family. The present checker permits the entire displayed family.

Theorem 4.2 (Candidate count; `candidates_length`)**.** *The candidate list has length $|R|L$.*

This is a search-space count. Repeated identifiers, exhausted claims or an uninformative execution stream can give redundant candidates or indistinguishable futures. The theorem neither proves mL distinct behavioral classes for arbitrary R nor imposes a universal $\lceil \log_2(mL) \rceil$ information requirement.

Definition 4.3 (Replay and decision)**.** The function $\texttt{replay}\, L\, n\, \mathrm{ids}\, st\, \vec{x}$ runs the quantities of $\vec{x}$ in sequence and emits one identifier-aligned fill vector per execution. Writing $\mathcal{C} = \texttt{candidates}\, L\, R$, the checker returns true exactly when

$$\bigvee_{st \in \mathcal{C}} \left[\texttt{replay}\, L\, n\, (\mathrm{ids}\, R)\, st\, \vec{x} = \mathrm{obs} \right].$$

Its Lean name is `wheelConsistent`.

The theorems quantify over the supplied fuel n and therefore compare the supplied fuel-bounded semantics. For complete executions with $L > 0$, use $n = 1 + \sum_i r_i$ from the initial remaining claims. The companion descent bound suffices because each positive step allocates at least one share, remaining claims only decrease, and each candidate allowance is positive. An input exceeding the remaining total leaves an unfilled residual. No arrivals, cancellations or changes of participant priority are introduced during this replay window.

Theorem 4.4 (Soundness; `wheelConsistent_sound`)**.** *Acceptance supplies a state $st \in \mathcal{C}$ whose replay equals the entire observation sequence.*

Theorem 4.5 (Family-relative completeness; `wheelConsistent_complete`)**.** *If a member of $\mathcal{C}$ replays to the observations, the checker accepts.*

Theorem 4.6 (Acceptance from initialization; `wheelConsistent_honest`)**.** *If $L > 0$ and $|R| > 0$, replay from $\texttt{initW}\, L\, R$ is accepted, for every quantity stream and fuel.*

Theorem 4.7 (Prefix monotonicity; `wheelConsistent_prefix`)**.** *If the checker accepts $\vec{x} +\!\!+ \vec{y}$ with observations $o_1 +\!\!+ o_2$, and $|o_1| = |\vec{x}|$, it accepts $\vec{x}$ with o_1.*

Proof sketch. Soundness and completeness unfold the finite disjunction. The initialized state is a candidate by `initW_mem_candidates`. The replay of a concatenated stream splits into the first replay and a replay from its terminal state (`replay_append`). The length identity `replay_length` then identifies the accepted prefix. □

Corollary 4.8 (Filtering)**.** *Let $\mathcal{C}_k$ be the candidates explaining the first k observed executions. Then $\mathcal{C}_0 \supseteq \mathcal{C}_1 \supseteq \cdots$. If the observations are generated by a member of the family, that member remains in every $\mathcal{C}_k$. An empty set excludes all members under the supplied inputs.*

Example 4.9 (Identification and ambiguity)**.** Take claims $(700, 500, 900, 600, 800)$ and $L = 100$. Replay quantities $(250, 130, 420, 90, 310, 260)$ from the initialized state with fuel 3501. The successive survivor counts are

$$500, \quad 1, \quad 1, \quad 1, \quad 1, \quad 1, \quad 1.$$

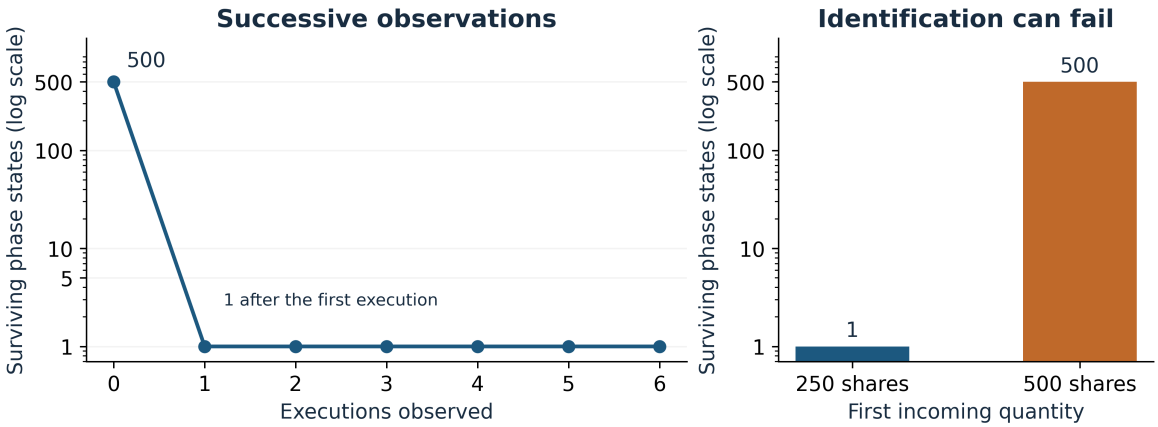

Figure 2: Finite filtering for Example 4.9. The left panel uses the six-execution stream; the right compares first quantities of 250 and 500 shares. A complete cycle can preserve pointer ambiguity. Counts are Lean evaluations of the actual replay checker.

The first fill $(100, 100, 50, 0, 0)$ selects participant 1 with full allowance. This is an evaluated example, not a generic one-execution identification theorem. On the same claims, a first execution of 500 shares instead yields $(100, 100, 100, 100, 100)$ and leaves all 500 candidates: every rotation and allowance. These values are exported by the included Lean script.

The three-participant kernel-checked instances `ex_honest` and `ex_reset_rejected` use claims $(300, 300, 200)$ and quantities $(250, 150)$. The persistent record is accepted; the reset record $((100, 100, 50), (100, 50, 0))$ is rejected by every candidate.

Remark 4.10 (Computational scope). The direct procedure performs at most mL replays; their costs also depend on fuel, stream length and identifier lookup. No production-scale runtime benchmark is claimed. Participant attribution and a stable eligible queue are separate requirements that this finite search cannot reconstruct.

5 Clock uncertainty with order identities

Let δ express a declared bound on unresolved ordering uncertainty, after available sequencing evidence has been used. It need not equal a feed's timestamp unit. A tied timestamp in a trusted sequenced stream does not erase priority; an uncertainty neighbourhood should include only queues still consistent with the evidence. The finite model below takes its neighbourhood as an input assumption.

Definition 5.1 (Labelled queues and swap neighbourhoods)**.** A queue entry is a triple (τ, i, q) of timestamp, unique order identifier and remaining size. Let I be a fixed list containing those identifiers once each. Define $\mathrm{LPT}_I(Q, x)$ by computing price–time fills in queue order and restoring them to the fixed order I. This is `labelledPriceTime`.

The function `swaps1` $\delta\, Q$ lists queues obtained by one adjacent swap with timestamp gap at most δ. The function `nbhd` $\delta\, k\, Q$ lists queues reachable by at most k such swaps, including Q. The clock checker is

$$\mathrm{Conforms}_{\delta,k}(I, Q, A, x) = \bigvee_{Q' \in \texttt{nbhd}\,\delta\,k\,Q} [A = \mathrm{LPT}_I(Q', x)].$$

Its Lean name is `conformsPT`δ; the identifier list and observed fills stay fixed while the candidate queue changes. Natural-number timestamp gaps are encoded as both $\tau_a - \tau_b \le \delta$ and $\tau_b - \tau_a \le \delta$.

Theorem 5.2 (Monotonicity in uncertainty; `conformsPTδ_mono`)**.** *If $\delta \le \delta'$, acceptance at (δ, k) implies acceptance at (δ', k), with I, Q, A, x fixed.*

Proof sketch. An allowed adjacent swap remains allowed when the bound grows (`swaps1_mono`). Induction on k gives neighbourhood inclusion (`nbhd_mono`); the same explaining queue remains a witness. □

Proposition 5.3 (Exact acceptance; `conformsPTδ_of_conformsPT`)**.** *If $A = \mathrm{LPT}_I(Q, x)$, the clock checker accepts for every δ, k.*

Theorem 5.4 (Sufficient adjacent masking; `jump_masked`)**.** *Let $Q = U +\!\!+ [a, b] +\!\!+ V$, let Q' swap a, b, and suppose their timestamp gap is at most δ. Then the checker accepts $\mathrm{LPT}_I(Q', x)$ at $(\delta, 1)$ against Q.*

This statement supplies an alternative explanation. To call the observed record a deviation from the original allocation additionally requires $\mathrm{LPT}_I(Q', x) \neq \mathrm{LPT}_I(Q, x)$. For a longer queue, the theorem does not give a converse: other allowed queues may explain the same fills.

Theorem 5.5 (Exact two-order threshold; `twoOrder_masking_iff`)**.** *Let $Q = [a, b]$, with $\mathrm{LPT}_I([b, a], x) \neq \mathrm{LPT}_I([a, b], x)$. The checker accepts the swapped record $\mathrm{LPT}_I([b, a], x)$ at $(\delta, 1)$ if and only if $|\tau_a - \tau_b| \le \delta$.*

Proof. There are only two possible queues. Below the gap threshold the neighbourhood contains only $[a, b]$, whose fills differ by hypothesis. At or above the threshold it contains $[b, a]$, which explains the record. □

Example 5.6 (Equal quantities, different orders)**.** Orders A and B both claim 200 shares, with timestamps 10 and 12. An incoming quantity of 200 gives labelled fills $(200, 0)$ in order A,B and $(0, 200)$ in order B,A. The latter record is accepted at $\delta = 2$ and rejected at $\delta = 1$ (`ex_equal_size_clock_jump`). Their equal sizes do not make their identities interchangeable.

For the three-order queue $((10, 1, 200), (12, 2, 300), (40, 3, 100))$ and $x = 400$, the swapped record in identifier order $(1, 2, 3)$ is $(100, 300, 0)$. It is accepted at $\delta = 5$ and rejected at $\delta = 1$ (`ex_masked`, `ex_not_masked`).

Proposition 5.7 (Exponential-gap experiment)**.** *Consider two orders with distinct swapped allocations, and let their nonnegative time gap G have an exponential distribution with rate $\lambda > 0$. In the one-swap uncertainty experiment with real threshold $\delta \ge 0$, the masking probability is*

$$\Pr(G \le \delta) = 1 - e^{-\lambda\delta}.$$

Proof. The experiment masks precisely on its threshold event. Integration of the density $\lambda e^{-\lambda g}$ over $0 \le g \le \delta$ gives the expression. Adjacent gaps in a homogeneous Poisson process provide one source of this distribution. □

This calculation is ordinary real-valued probability, outside the Lean development. It uses the same threshold predicate as Theorem 5.5; Lean timestamps themselves are natural numbers. The expression is not a universal detection-power formula for exchange feeds, longer queues, selected violations or uncertain reconstruction. Quantization, dependence, selection of pairs and sequence evidence can change the applicable experiment.

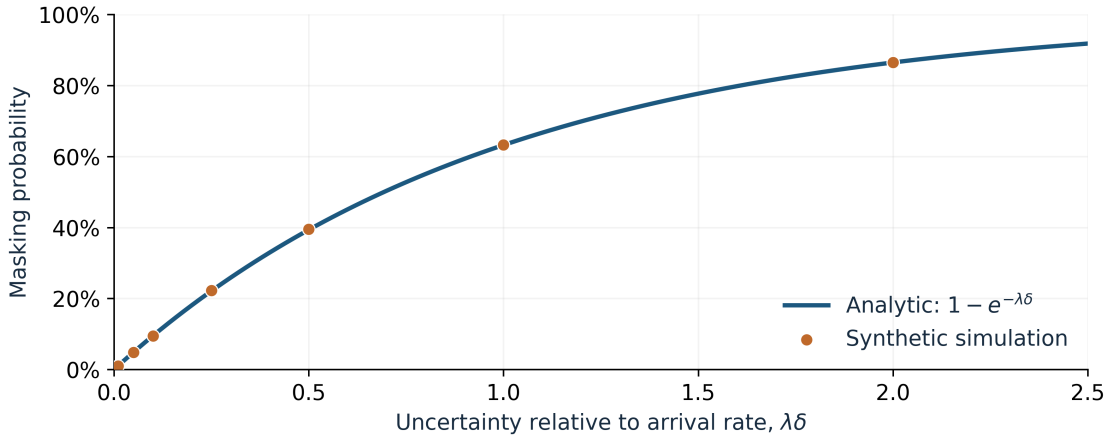

Figure 3: The exponential-gap experiment: analytic probability and simulated frequencies. The horizontal coordinate is the dimensionless product $\lambda\delta$. The simulation uses 200,000 exponential draws and seed 7; it contains no market observations.

Table 1: Reproducible synthetic masking frequencies, using one common sample of 200,000 unit-rate exponential gaps and Python's `random.Random(7)` generator. The source package includes the script and generated values.

$\lambda\delta$	Simulated frequency	$1 - e^{-\lambda\delta}$
0.01	0.0100	0.0100
0.05	0.0486	0.0488
0.10	0.0948	0.0952
0.25	0.2224	0.2212
0.50	0.3949	0.3935
1.00	0.6327	0.6321
2.00	0.8645	0.8647

6 Audit protocol and interpretation

The logical decision and the quality of its inputs are different obligations. The following protocol keeps the conclusion attached to both.

Protocol 6.1 (Model-relative conformance audit)**.** 1. **Declare the model.** Identify the rule, eligible order set, observation projection and initial-state family. For clock uncertainty, state δ, swap budget k, and the evidence permitting each ambiguity. Retain trusted sequence information.

2. **Reconstruct the input.** Supply the incoming quantity, remaining claims, stable identities, priority evidence and aligned execution fills. Account for loss, duplicates, cancellations, replacement, hidden liquidity and rule-specific eligibility. For a wheel, supply the cyclic participant order and attribution at the rule's granularity.

3. **Run the decision.** Use `conformsPT`, `conformsPT`δ or `wheelConsistent`. Fix sufficient fuel for complete wheel executions. Preserve inputs and the artifact identifier with the result.

4. **Report the finding.** A rejection means that no declared admissible explanation matches

Table 2: Scope of the established comparisons. Clock uncertainty and latent-wheel filtering are separate checkers; their combination is not implemented here.

Case	Total only	Exact depth	Clock uncertainty
Positive price–time fill transfer	Same total	Rejected for fixed honest queue	Adjacent swap: sufficient masking; two-order converse needs distinct fills
Wrong aggregate total	Rejected if expected total is known	Rejected under same input contract	Rejected when full unique labels and expected total are supplied
Wheel reset	Total can agree	Rejected in the stated instance	Not combined here
Unknown wheel phase	No phase identification from total alone in general	Candidate filtering	Not combined here
Executions with identical observed fills	Same projection	Indistinguishable by fills	Same labelled observation

those inputs. Preserve the mismatching allocation or the exhausted candidate search. An acceptance means "compatible with the declared model." Input-validation failures remain a separate outcome.

5. **Separate population inference.** Counts are descriptive unless a sampling model is justified. A Clopper–Pearson interval applies to independent Bernoulli trials with a common probability (Clopper and Pearson, 1934); consecutive executions do not automatically satisfy that assumption. Dependent or selected samples need an appropriate separate analysis.

Logical error boundary. Let H be the declared conforming execution family and C an observation map. The ideal compatibility test accepts z exactly when $z \in C(H)$. If the actual execution is in H and the observed record is its exact image, rejection is impossible. This is a conditional logical guarantee, not an unconditional claim about an empirical false-positive rate. Among executions outside H, the unresolved set is precisely

$$\{e \notin H : \exists h \in H,\ C(e) = C(h)\}.$$

It is the union of observation classes meeting H, restricted to nonconforming executions. This expression depends jointly on the observation map and the conforming family. This set identity follows directly from the compatibility definition; it is not an additional Lean declaration. Wrong reconstruction, an excluded legitimate state or a misspecified rule can invalidate an application even when the decision procedure is correct.

Data contract. An order-level replay needs order reference, price, side, remaining size, executed size and the ordering evidence relevant to priority. ITCH's displayed-order identifiers and sequenced architecture supply parts of this contract, not a complete model of all hidden or exceptional matching. A wheel audit additionally needs the participant categories and aggregation conventions used by the rule. Public trade totals do not supply this information. The package contains no parser certifying the transformation from any public feed to these inputs.

7 Limits and empirical application

The finite wheel family fixes claims and cyclic order at the audit boundary. It allows unknown phase and rebased counters, not arbitrary arrivals, cancellations or changing eligibility during replay. Completeness is relative to that family and the supplied fuel. Historical information can restrict the family; absent information cannot be recovered by declaring an audit result.

The clock model permits bounded adjacent swaps. Its combinatorial neighbourhood is an explicit uncertainty assumption, not a statistical description of a particular feed. The two-order converse does not characterize all longer-queue collisions. A full venue model would also connect transport sequencing, timestamp generation and priority rules to the admissible neighbourhood. This connection is not formalized here.

All examples are synthetic. The Lean declarations establish deductive properties; the plotted candidate counts are evaluations of the executable definitions; the probability table is a reproducible simulation of a stated distribution. None supplies an empirical violation rate. Applying the protocol would require a validated reconstruction layer and documented sampling design before reporting venue- or symbol-level findings. The regulatory traceability modules inherited from Paper I are supporting source, not new legal results of this paper.

8 Formal artifact and reproducibility

`TapeConformance.lean` contains 29 theorem declarations within a complete development of 391. Lean 4.22.0 checks the package using the core library only, with no Mathlib dependency. The source audit rejects proof placeholders, and `Check.lean` prints the dependencies of every counted declaration. None of the 29 new declarations depends on `Classical.choice`; 48 inherited declarations do. Of the seven new worked-instance declarations, six depend on no axioms; `ex_candidates` depends on propositional extensionality. Counts include elementary lemmas and evaluated examples, not 391 independent research contributions.

Table 3 separates these dependencies. The probabilistic calculation, observation-class identity and binary-encoding discussion are mathematical exposition, not claims of additional Lean formalization. The three PNG figures have included inputs and regeneration scripts. The finite figure data are exported from Lean and checked against an independent Python implementation; the simulation records its generator, seed and sample size.

Availability and citation. This paper is identified by DOI 10.5281/zenodo.22392230. The accompanying package includes the Lean development, LaTeX source, figures, data, build instructions, citation metadata and file hashes. The GitHub repository for the source code is `https://github.com/MiquelNoguerAlonso/Formal-Verification`. The companion paper has the separate DOI 10.5281/zenodo.22343804.

Table 3: All 29 declarations in `TapeConformance.lean`. Here p denotes `propext` and q denotes `Quot.sound`.

Group	Declaration(s)	Axioms
Total/depth	`sum_app`	p
	`jump_tape_invisible`, `jump_depth_detected`, `honest_conforms`	pq
Candidates	`rotations_length`, `length_flatMap_const`, `candidates_length`	p
	`rotations_self`, `initW_mem_candidates`	pq
Replay	`replay_length`, `replay_append`	p
	`wheelConsistent_sound`, `wheelConsistent_complete`, `wheelConsistent_honest`, `wheelConsistent_prefix`	pq
Clock	`swaps1_mono`, `nbhd_mono`, `conformsPT`δ`_mono`, `conformsPT`δ`_of_conformsPT`, `swap_mem_swaps1`, `jump_masked`, `twoOrder_masking_iff`	pq
Instances	`ex_candidates`	p
	`ex_honest`, `ex_reset_rejected`, `ex_tape_jump`, `ex_masked`, `ex_not_masked`, `ex_equal_size_clock_jump`	none

References

Matteo Aquilina, Eric Budish, and Peter O'Neill. Quantifying the high-frequency trading "arms race". *Quarterly Journal of Economics*, 137(1):493–564, 2022.

Robert P. Bartlett and Justin McCrary. How rigged are stock markets? evidence from microsecond timestamps. *Journal of Financial Markets*, 45:37–60, 2019.

Paul Brennan. An introduction to Imandra Markets. Imandra Markets technical case study; https://medium.com/imandra/an-introduction-to-imandra-markets-d9b7419916f2, 2024. Published February 20, 2024; describes exchange digital twins, matching-logic changes, design verification, and model-based validation.

C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934.

Shengwei Ding, John Hanna, and Terrence Hendershott. How slow is the NBBO? a comparison with direct exchange feeds. *Financial Review*, 49(2):313–332, 2014.

Craig W. Holden and Stacey Jacobsen. Liquidity measurement problems in fast, competitive markets: Expensive and cheap solutions. *Journal of Finance*, 69(4):1747–1785, 2014.

Andrei Kirilenko, Albert S. Kyle, Mehrdad Samadi, and Tugkan Tuzun. The flash crash: High-frequency trading in an electronic market. *Journal of Finance*, 72(3):967–998, 2017.

Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

Hervé Moulin. Priority rules and other asymmetric rationing methods. *Econometrica*, 68(3):643–684, 2000.

Nasdaq. Nasdaq TotalView-ITCH 5.0 specification. Nasdaq Trader technical specification, n.d. URL https://www.nasdaqtrader.com/content/technicalsupport/specifications/dataproducts/NQTVITCHspecification.pdf. Architecture and data types, pp. 3–4; displayed-order messages, pp. 13–15. Accessed September 5, 2026.

Miquel Noguer i Alonso. Foundations of formal market microstructure in U.S. equities: Machine-checked allocation, composition, necessary state, and regulatory traceability. Formal Market Microstructure, Paper I. Artificial Intelligence Finance Institute, 2026. URL https://doi.org/10.5281/zenodo.22343804.

NYSE. Daily TAQ client specification, version 4.0. NYSE technical specification, November 2, 2022. URL https://www.nyse.com/publicdocs/nyse/data/Daily_TAQ_Client_Spec_v4.0.pdf. Product description, p. 4; trade-record fields, pp. 17–20. Accessed September 5, 2026.

Grant O. Passmore and Denis Ignatovich. Formal verification of financial algorithms. In *Automated Deduction – CADE 26*, volume 10395 of *LNCS*, pages 26–41. Springer, 2017.

William Thomson. Axiomatic and game-theoretic analysis of bankruptcy and taxation problems: a survey. *Mathematical Social Sciences*, 45(3):249–297, 2003.